In [1]:
```
import pandas as pd

portfolio = pd.read_csv("bond_portfolio_data.csv")
yield_curve = pd.read_csv("yield_curve_history.csv")
scenarios = pd.read_csv("monte_carlo_scenarios.csv")
portfolio.head()
portfolio.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 44 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   BondID                    300 non-null    str
 1   ISIN                      300 non-null    str
 2   Issuer                    300 non-null    str
 3   Sector                    300 non-null    str
 4   CreditRating              300 non-null    str
 5   Currency                  300 non-null    str
 6   FaceValue                 300 non-null    int64
 7   CouponRate                300 non-null    float64
 8   CouponFrequency           300 non-null    int64
 9   BondType                  300 non-null    str
 10  IssueDate                 300 non-null    str
 11  MaturityDate              300 non-null    str
 12  ValuationDate             300 non-null    str
 13  YearsToMaturity           300 non-null    float64
 14  OriginalMaturityYears     300 non-null    int64
 15  YieldToMaturity           300 non-null    float64
 16  CleanPrice                300 non-null    float64
 17  AccruedInterest           300 non-null    float64
 18  DirtyPrice                300 non-null    float64
 19  MacaulayDuration          300 non-null    float64
 20  ModifiedDuration          300 non-null    float64
 21  Convexity                 300 non-null    float64
 22  DV01_Per100Face           300 non-null    float64
 23  EffectiveDuration         300 non-null    float64
 24  EffectiveConvexity        300 non-null    float64
 25  SpreadOverBenchmark_bps   300 non-null    int64
 26  OAS_bps                   300 non-null    int64
 27  ZSpread_bps               300 non-null    int64
 28  KeyRateBucket             300 non-null    str
 29  Quantity                  300 non-null    int64
 30  MarketValue_LocalCcy      300 non-null    float64
 31  MarketValue_INR           300 non-null    float64
 32  PortfolioWeight           300 non-null    float64
 33  IsCallable                300 non-null    bool
 34  CallDate                  17 non-null     str
 35  IsPutable                 300 non-null    bool
 36  IsFloatingRate            300 non-null    bool
 37  BenchmarkIndex            300 non-null    str
 38  PriceChange_Up50bps       300 non-null    float64
 39  PriceChange_Dn50bps       300 non-null    float64
```

```
 40  PriceChange_Up100bps     300 non-null    float64
 41  PriceChange_Dn100bps     300 non-null    float64
 42  PriceChange_Up200bps     300 non-null    float64
 43  PriceChange_Dn200bps     300 non-null    float64
dtypes: bool(3), float64(21), int64(7), str(13)
memory usage: 97.1 KB
```

In [2]:
```
portfolio["MarketValue_INR"].sum()
```

Out[2]:
```
np.float64(32059345.970000003)
```

In [3]:
```
portfolio_duration = (
portfolio["ModifiedDuration"] *
portfolio["MarketValue_INR"]
).sum() / portfolio["MarketValue_INR"].sum()
```

In [4]:
```
portfolio_convexity = (
portfolio["Convexity"] *
portfolio["MarketValue_INR"]
).sum() / portfolio["MarketValue_INR"].sum()
```

In [5]:
```
portfolio["DV01"] = (
portfolio["ModifiedDuration"]
*
portfolio["DirtyPrice"]
*
0.0001
)
```

In [6]:
```
import matplotlib.pyplot as plt

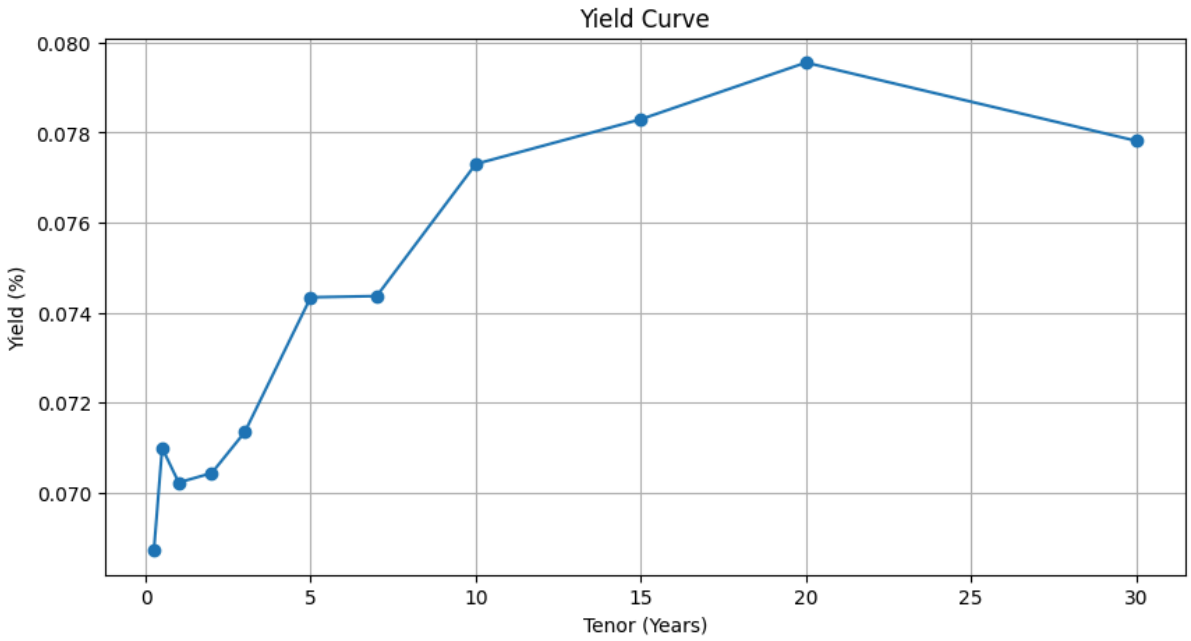
latest_curve = yield_curve[
    yield_curve["CurveDate"] ==
    yield_curve["CurveDate"].max()
]

plt.figure(figsize=(10,5))

plt.plot(
    latest_curve["Tenor_Years"],
    latest_curve["Yield"],
    marker='o'
)

plt.title("Yield Curve")
plt.xlabel("Tenor (Years)")
plt.ylabel("Yield (%)")
plt.grid(True)

plt.show()
```



In [7]:
```
print(portfolio.columns.tolist())
portfolio.head()
```

```
['BondID', 'ISIN', 'Issuer', 'Sector', 'CreditRating', 'Currency',
'FaceValue', 'CouponRate', 'CouponFrequency', 'BondType', 'IssueDate',
'MaturityDate', 'ValuationDate', 'YearsToMaturity', 'OriginalMaturityYears',
'YieldToMaturity', 'CleanPrice', 'AccruedInterest', 'DirtyPrice',
'MacaulayDuration', 'ModifiedDuration', 'Convexity', 'DV01_Per100Face',
'EffectiveDuration', 'EffectiveConvexity', 'SpreadOverBenchmark_bps',
'OAS_bps', 'ZSpread_bps', 'KeyRateBucket', 'Quantity',
'MarketValue_LocalCcy', 'MarketValue_INR', 'PortfolioWeight', 'IsCallable',
'CallDate', 'IsPutable', 'IsFloatingRate', 'BenchmarkIndex',
'PriceChange_Up50bps', 'PriceChange_Dn50bps', 'PriceChange_Up100bps',
'PriceChange_Dn100bps', 'PriceChange_Up200bps', 'PriceChange_Dn200bps',
'DV01']
```

Out[7]: Columns 1–8 of 22

	BondID	ISIN	Issuer	Sector	CreditRating	Currency	FaceValue
0	ZTFI-0001	IN1043321819	Government of India	Government	SOV	INR	100
1	ZTFI-0002	IN7940265423	Government of India	Government	SOV	INR	100
2	ZTFI-0003	IN7816184959	Government of India	Government	SOV	INR	100
3	ZTFI-0004	IN4752553419	Government of India	Government	SOV	INR	100
4	ZTFI-0005	IN5030564139	Government of India	Government	SOV	INR	100

Columns 9–15 of 22 · first column repeated

	CouponRate	CouponFrequency	BondType	...	IsPutable	IsFloatingRate	BenchmarkIndex
0	0.0620	2	Fixed	...	False	False	IN10YT
1	0.0773	2	Fixed	...	False	False	IN10YT
2	0.0555	2	Fixed	...	False	False	IN10YT
3	0.0681	2	Fixed	...	False	False	IN10YT
4	0.0750	2	Fixed	...	False	False	IN10YT

Columns 16–22 of 22 · first column repeated

	PriceChange_Up50bps	PriceChange_Dn50bps	PriceChange_Up100bps	PriceChange_Dn100bps	PriceChange_Up200bps	PriceChange_Dn200bps	DV01
0	0.9207	-0.9098	1.8522	-1.8086	3.7480	-3.5736	0.018361
1	6.1042	-5.6378	12.6747	-10.8091	27.2150	-19.7526	0.118674
2	2.0557	-2.0040	4.1631	-3.9562	8.5332	-7.7056	0.041104
3	6.3440	-5.7961	13.2358	-11.0444	28.6631	-19.8973	0.123282
4	5.4267	-5.0533	11.2267	-9.7333	23.9468	-17.9731	0.105296

5 rows × 45 columns

In [8]:
```
total_market_value = portfolio["MarketValue_INR"].sum()

print(f"Portfolio Market Value: ₹{total_market_value:,.2f}")
```

```
Portfolio Market Value: ₹32,059,345.97
```

In [9]:
```
portfolio_duration = (
    portfolio["ModifiedDuration"]
    * portfolio["MarketValue_INR"]
).sum() / portfolio["MarketValue_INR"].sum()

print("Portfolio Duration:", round(portfolio_duration,2))
```

```
Portfolio Duration: 4.2
```

In [10]:
```
portfolio_convexity = (
    portfolio["Convexity"]
    * portfolio["MarketValue_INR"]
).sum() / portfolio["MarketValue_INR"].sum()

print("Portfolio Convexity:", round(portfolio_convexity,2))
```

```
Portfolio Convexity: 44.06
```

In [11]:
```
portfolio["DV01_INR"] = (
    portfolio["ModifiedDuration"]
    *
    portfolio["DirtyPrice"]
    *
    portfolio["Quantity"]
    *
    0.0001
)

total_dv01 = portfolio["DV01_INR"].sum()

print(f"Portfolio DV01: ₹{total_dv01:,.2f}")
```

```
Portfolio DV01: ₹902,038.45
```

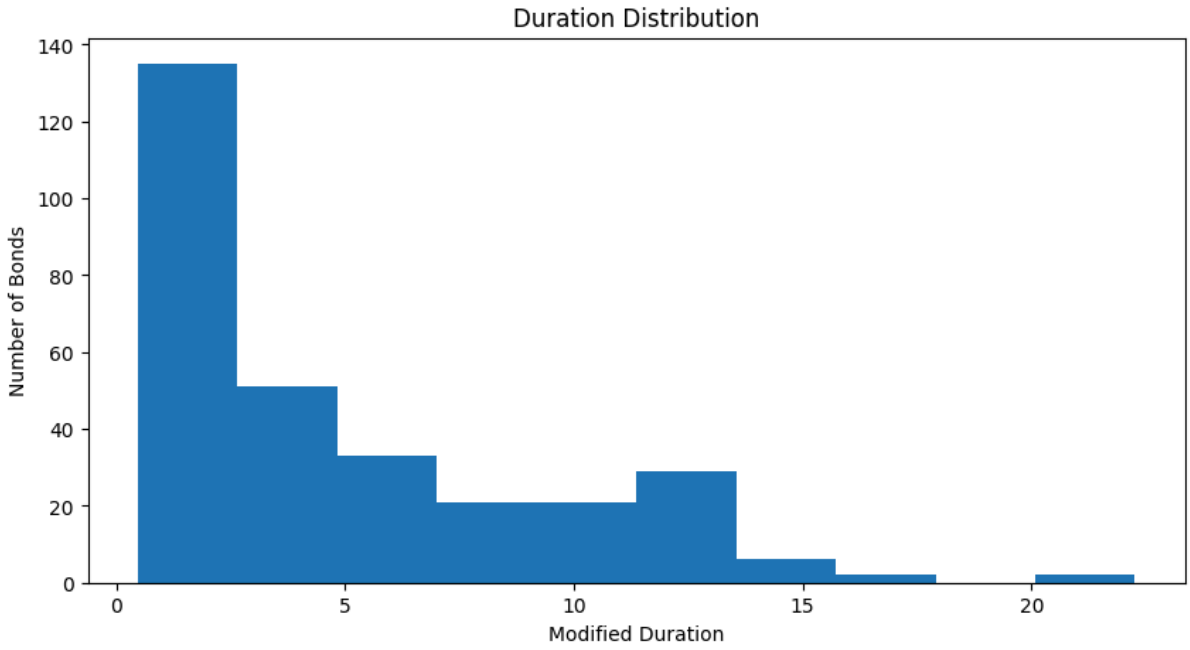
In [12]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))

plt.hist(
    portfolio["ModifiedDuration"],
    bins=10
)

plt.title("Duration Distribution")
plt.xlabel("Modified Duration")
plt.ylabel("Number of Bonds")

plt.show()
```



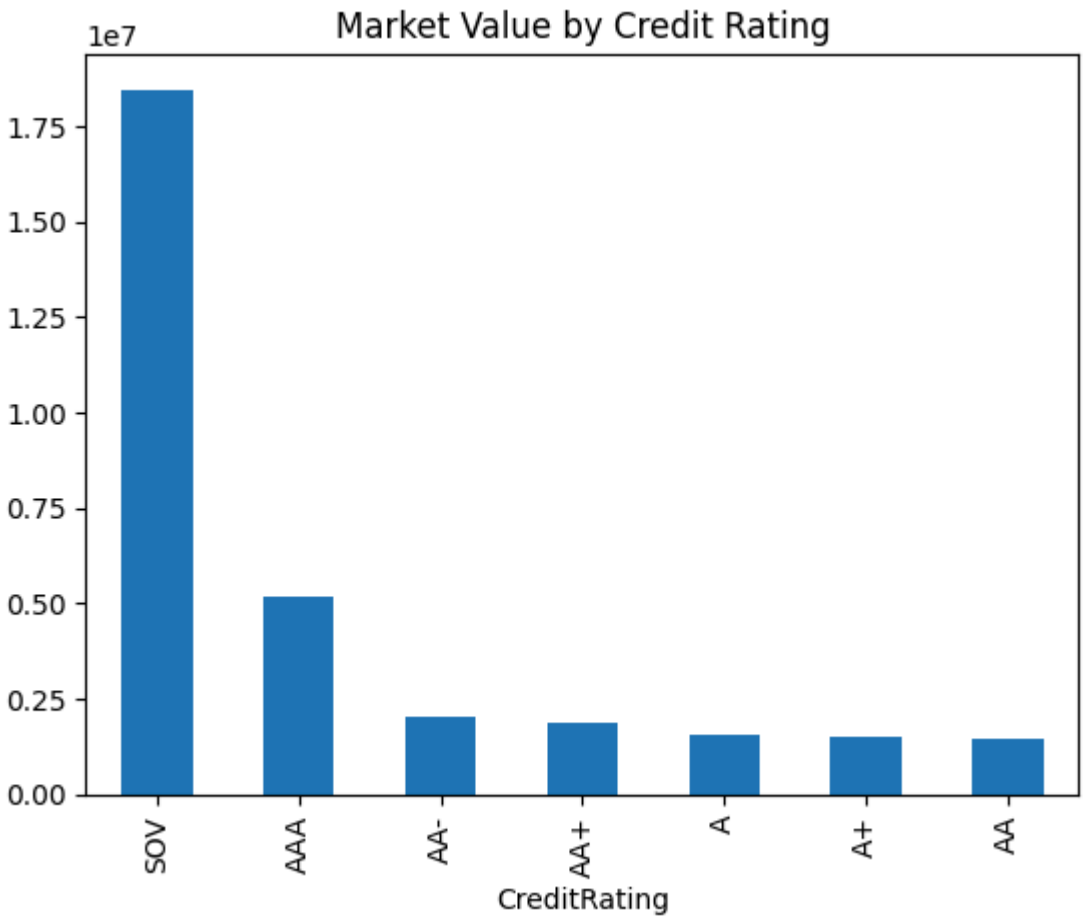
In [13]:
```
rating_risk = (
    portfolio.groupby("CreditRating")
    ["MarketValue_INR"]
    .sum()
    .sort_values(ascending=False)
)

print(rating_risk)
```

```
CreditRating
SOV    18452107.18
AAA     5163705.40
AA-     2042166.68
AA+     1858230.75
A       1579567.44
A+      1498636.19
AA      1464932.33
Name: MarketValue_INR, dtype: float64
```

In [14]:
```
rating_risk.plot(
    kind="bar"
)

plt.title("Market Value by Credit Rating")
plt.show()
```

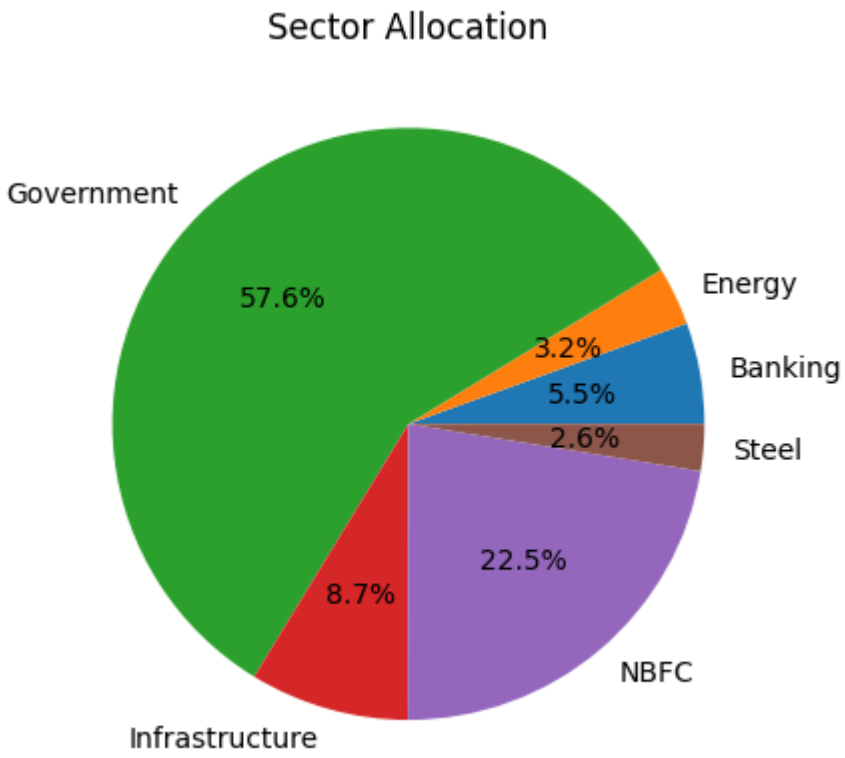


In [15]:
```
sector_exposure = (
    portfolio.groupby("Sector")
    ["MarketValue_INR"]
    .sum()
)

sector_exposure.plot(
    kind="pie",
    autopct="%1.1f%%"
)

plt.ylabel("")
plt.title("Sector Allocation")

plt.show()
```



In [16]:
```
dy = 0.005

portfolio["Shock50bp"] = (
    -portfolio["ModifiedDuration"]*dy
    +
    0.5*
    portfolio["Convexity"]*
    (dy**2)
)
```

In [17]:
```
dy = 0.01

portfolio["Shock100bp"] = (
    -portfolio["ModifiedDuration"]*dy
    +
    0.5*
    portfolio["Convexity"]*
    (dy**2)
)
```

In [18]:
```
dy = 0.02

portfolio["Shock200bp"] = (
    -portfolio["ModifiedDuration"]*dy
    +
    0.5*
    portfolio["Convexity"]*
    (dy**2)
)
```

In [19]:
```
stress_loss_100bp = (
    portfolio["Shock100bp"]
    *
    portfolio["MarketValue_INR"]
).sum()

print(
    f"Portfolio Loss (+100bps): ₹{stress_loss_100bp:,.2f}"
)
```

```
Portfolio Loss (+100bps): ₹-1,276,866.58
```

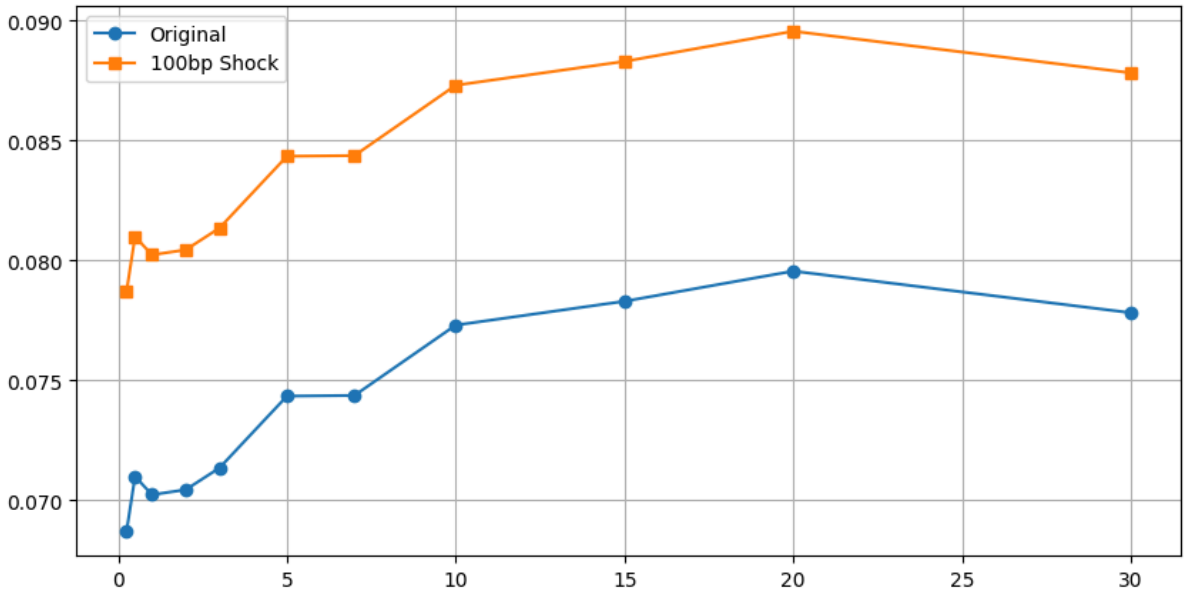
In [20]:
```
yield_curve["Yield_Shock100bp"] = (
    yield_curve["Yield"] + 0.01
)
plt.figure(figsize=(10,5))

latest_curve = yield_curve[
    yield_curve["CurveDate"] ==
    yield_curve["CurveDate"].max()
]

plt.plot(
    latest_curve["Tenor_Years"],
    latest_curve["Yield"],
    marker='o',
    label='Original'
)

plt.plot(
    latest_curve["Tenor_Years"],
    latest_curve["Yield"]+0.01,
    marker='s',
    label='100bp Shock'
)

plt.legend()
plt.grid(True)
plt.show()
```



In [21]:
```
portfolio_duration = (
    portfolio["ModifiedDuration"] *
    portfolio["MarketValue_INR"]
).sum() / portfolio["MarketValue_INR"].sum()

print("Portfolio Duration:", round(portfolio_duration, 2))
```

```
Portfolio Duration: 4.2
```

In [22]:
```
portfolio_convexity = (
    portfolio["Convexity"] *
    portfolio["MarketValue_INR"]
).sum() / portfolio["MarketValue_INR"].sum()

print("Portfolio Convexity:", round(portfolio_convexity, 2))
```

```
Portfolio Convexity: 44.06
```

In [23]:
```
portfolio["DV01_INR"] = (
    portfolio["ModifiedDuration"]
    * portfolio["DirtyPrice"]
    * portfolio["Quantity"]
    * 0.0001
)

total_dv01 = portfolio["DV01_INR"].sum()

print(f"Portfolio DV01: ₹{total_dv01:,.2f}")
```

```
Portfolio DV01: ₹902,038.45
```

In [24]:
```
dy = 0.01

portfolio["Shock100bp"] = (
    -portfolio["ModifiedDuration"] * dy
    +
    0.5 * portfolio["Convexity"] * (dy**2)
)

stress_loss_100bp = (
    portfolio["Shock100bp"]
    * portfolio["MarketValue_INR"]
).sum()

print(f"Portfolio Loss (+100bps): ₹{stress_loss_100bp:,.2f}")
```

```
Portfolio Loss (+100bps): ₹-1,276,866.58
```

In [25]:
```
import numpy as np

n_scenarios = 10000

rate_shocks = np.random.normal(
    loc=0,
    scale=0.01,
    size=n_scenarios
)
portfolio_pnl = (
    -portfolio_duration * rate_shocks
    +
    0.5 * portfolio_convexity * (rate_shocks**2)
)

portfolio_pnl = (
    portfolio_pnl
    * portfolio["MarketValue_INR"].sum()
)
```

In [26]:
```
var95 = np.percentile(portfolio_pnl, 5)

print(f"95% VaR: ₹{var95:,.2f}")
```

```
95% VaR: ₹-2,005,845.82
```

In [27]:
```
es95 = portfolio_pnl[
    portfolio_pnl ⩽ var95
].mean()

print(f"Expected Shortfall: ₹{es95:,.2f}")
```

```
Expected Shortfall: ₹-2,472,004.42
```

In [28]:
```
import matplotlib.pyplot as plt

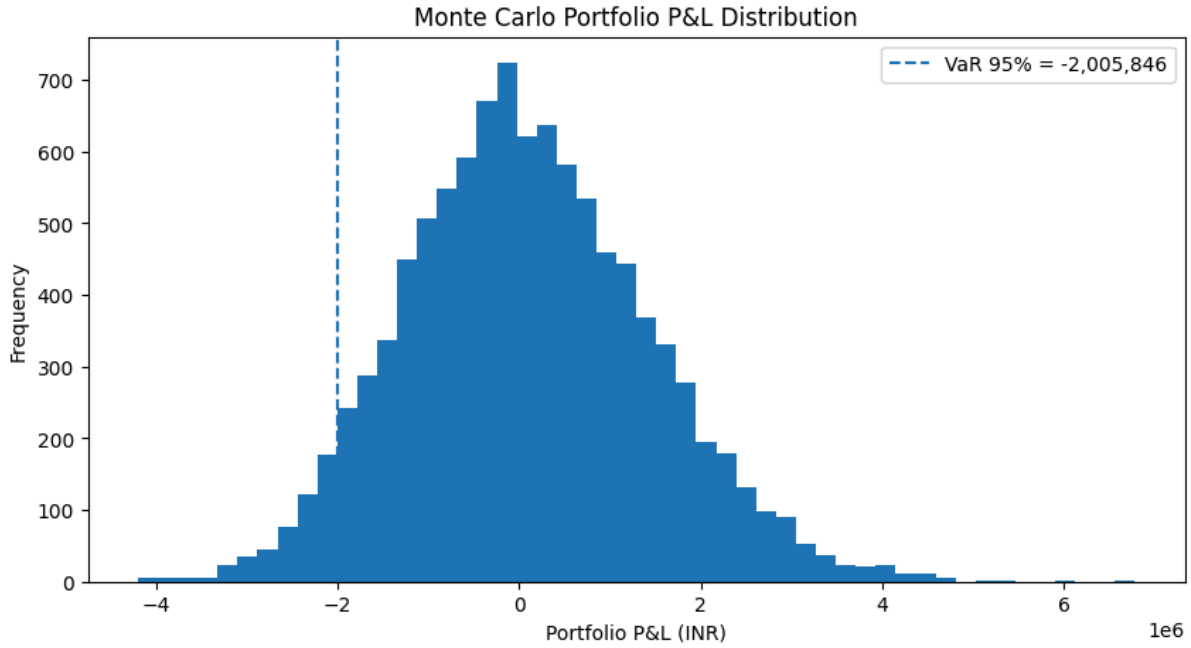
plt.figure(figsize=(10,5))

plt.hist(
    portfolio_pnl,
    bins=50
)

plt.axvline(
    var95,
    linestyle="--",
    label=f"VaR 95% = {var95:,.0f}"
)

plt.title("Monte Carlo Portfolio P&L Distribution")
plt.xlabel("Portfolio P&L (INR)")
plt.ylabel("Frequency")
plt.legend()

plt.show()
```



In [29]:
```
portfolio["RiskScore"] = (
    portfolio["ModifiedDuration"] * 0.4
    +
    portfolio["Convexity"] * 0.3
    +
    portfolio["YieldToMaturity"] * 100 * 0.2
    +
    portfolio["OAS_bps"] * 0.1
)
```

In [30]:
```
portfolio[
    ["BondID","RiskScore"]
].head()
```

Out[30]:

	BondID	RiskScore
0	ZTFI-0001	3.40833
1	ZTFI-0002	56.96687
2	ZTFI-0003	9.58956
3	ZTFI-0004	70.76837
4	ZTFI-0005	48.12679

In [31]:
```
features = [
    "ModifiedDuration",
    "Convexity",
    "YieldToMaturity",
    "OAS_bps",
    "ZSpread_bps",
    "SpreadOverBenchmark_bps",
    "PortfolioWeight"
]
```

In [32]:
```
from sklearn.model_selection import train_test_split

X = portfolio[features]
y = portfolio["RiskScore"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

In [33]:
```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(
    n_estimators=200,
    random_state=42
)

rf.fit(X_train, y_train)

rf_predictions = rf.predict(X_test)
```

In [34]:
```
from sklearn.metrics import r2_score

print(
    "Random Forest R²:",
    round(
        r2_score(
            y_test,
            rf_predictions
        ),
        4
    )
)
```

```
Random Forest R²: 0.8653
```

In [35]:
```
import pandas as pd

importance = pd.DataFrame({
    "Feature": features,
    "Importance": rf.feature_importances_
})

importance.sort_values(
    by="Importance",
    ascending=False
)
```

Out[35]:

	Feature	Importance
1	Convexity	0.576999
0	ModifiedDuration	0.380111
3	OAS_bps	0.023891
5	SpreadOverBenchmark_bps	0.009679
4	ZSpread_bps	0.005180
2	YieldToMaturity	0.003150
6	PortfolioWeight	0.000990

In [36]:
```
import matplotlib.pyplot as plt

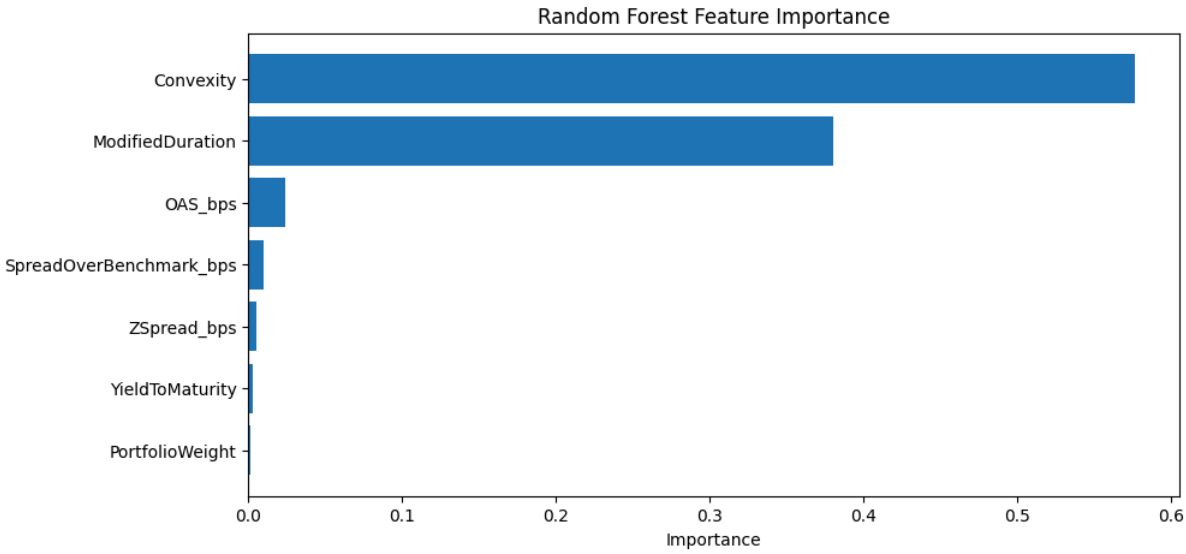
importance_sorted = importance.sort_values(
    by="Importance",
    ascending=True
)

plt.figure(figsize=(10,5))

plt.barh(
    importance_sorted["Feature"],
    importance_sorted["Importance"]
)

plt.title("Random Forest Feature Importance")
plt.xlabel("Importance")

plt.show()
```



In [37]:
```
from xgboost import XGBRegressor
from sklearn.metrics import r2_score

xgb = XGBRegressor(
    n_estimators=300,
    max_depth=4,
    learning_rate=0.05,
    random_state=42
)

xgb.fit(X_train, y_train)

xgb_predictions = xgb.predict(X_test)

xgb_r2 = r2_score(
    y_test,
    xgb_predictions
)

print("XGBoost R²:", round(xgb_r2,4))
```

```
XGBoost R²: 0.8957
```

In [38]:
```
comparison = pd.DataFrame({
    "Model":[
        "Random Forest",
        "XGBoost"
    ],
    "R2":[
        r2_score(y_test, rf_predictions),
        r2_score(y_test, xgb_predictions)
    ]
})

comparison
```

Out[38]:

	Model	R2
0	Random Forest	0.865340
1	XGBoost	0.895708

In [39]:
```
def bond_risk_agent(
    duration,
    convexity,
    var95
):

    if abs(var95) > 2000000:
        return "HIGH RISK"

    elif duration > 5:
        return "MODERATE RISK"

    else:
        return "LOW RISK"
risk_status = bond_risk_agent(
    portfolio_duration,
    portfolio_convexity,
    var95
)

print("Portfolio Risk Status:", risk_status)
```

```
Portfolio Risk Status: HIGH RISK
```

In [53]:
```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Dense
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

nn = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(32, activation="relu"),
    Dense(16, activation="relu"),
    Dense(8, activation="relu"),
    Dense(1)
])

nn.compile(
    optimizer='adam',
    loss='mse'
)

nn.fit(
    X_train_scaled,
    y_train,
    epochs=50,
    batch_size=16,
    verbose=0
)

nn_predictions = nn.predict(X_test_scaled)

nn_r2 = r2_score(
    y_test,
    nn_predictions
)

print("Neural Network R²:", round(nn_r2,4))
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step
Neural Network R²: 0.9977
```

In [41]:
```
portfolio_risk_score = (
    abs(var95)/100000
    +
    portfolio_duration*5
    +
    portfolio_convexity*0.5
)

print("Portfolio Risk Score:",
      round(portfolio_risk_score,2))
```

```
Portfolio Risk Score: 63.1
```

In [42]:
```
if portfolio_risk_score < 50:
    risk_band = "LOW"

elif portfolio_risk_score < 75:
    risk_band = "MODERATE"

else:
    risk_band = "HIGH"

print(risk_band)
```

```
MODERATE
```

In [63]:
```
from tensorflow.keras import Input

nn = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(32, activation='relu'),
    Dense(16, activation='relu'),
    Dense(8, activation='relu'),
    Dense(1)
])
```

In [64]:
```
portfolio.to_csv(
    "bond_risk_results.csv",
    index=False
)
```

In [65]:
```
print(type(portfolio_pnl))
print(len(portfolio_pnl))
print(portfolio_pnl[:5])
```

```
<class 'numpy.ndarray'>
10000
[  575190.50792366  -416026.22834986 -1671670.61770212  -507716.35751004
  1388497.32098656]
```

In [66]:
```
mc_results = pd.DataFrame({
    "Scenario": range(len(portfolio_pnl)),
    "PnL": portfolio_pnl
})

mc_results.head()
```

Out[66]:

	Scenario	PnL
0	0	5.751905e+05
1	1	-4.160262e+05
2	2	-1.671671e+06
3	3	-5.077164e+05
4	4	1.388497e+06

In [72]:
```
mc_results.to_csv("monte_carlo_results.csv", index=False)

print("Monte Carlo results exported successfully!")
```

```
Monte Carlo results exported successfully!
```

In [73]:
```
summary = pd.DataFrame({
    "Metric":[
        "Portfolio Market Value",
        "Average Duration",
        "Average Convexity",
        "VaR 95%",
        "Expected Shortfall",
        "Portfolio Risk Score"
    ],
    "Value":[
        portfolio["MarketValue_INR"].sum(),
        portfolio_duration,
        portfolio_convexity,
        var95,
        es95,
        portfolio_risk_score
    ]
})

summary.to_csv("portfolio_summary.csv",index=False)
```

In [74]:
```
comparison.to_csv(
    "model_comparison.csv",
    index=False
)
```

In [75]:
```
portfolio.to_csv(
    "portfolio_details.csv",
    index=False
)
```